

A group of four students are sitting at a table in a library, looking at a laptop screen. The background is filled with bookshelves. The image has a semi-transparent blue overlay on the left side and a semi-transparent red overlay on the right side.

Windows Forms- DB Processing

Recording the Main Transaction and the Inner Details of a Transaction for a Book Shop

About these notes!!

- These notes contain a demonstration of how to:
 - record a main Order transaction for a point of sale system (slides 1-16 – also provides a *soln to prac 6* – prototype test prac)
 - Record the inner details of a transaction (not required for the test but certainly required for the Major Project)

The Book Sale App

- The demonstration of the recording of the Inner Detail of a Transaction will be demonstrated via a Book App that is designed to record the sale of books; in this app the Customer details are not required
- The Book App consists of the following dataset structure:

BookOrder	
OrderNo	
OrderDate	
OrderAmt	
BookOrderTableAdapter	
Fill,GetData ()	

OrderDetail	
OrderNo	
BookTitle	
Quantity	
Price	
OrderDetailTableAda	
Fill,GetData ()	

Book	
BookTitle	
BookCost	
QOH	
BookTableAdapter	
Fill,GetData ()	

- Book Order are placed – and each Order consist of many Order Details and each OrderDetail references a single book – we will not implement the relationship in the database (for convenience of avoiding database generated errors)

The Interface

- Consists of many bits of functionality that is quite self-explanatory – the interface has options to:

- Search for a Book
- Update Book Stock (QOH)
- Add items to an Invoice
- Generate the total Amt owing
- Record the full Sale
 - i.e the total Sale Amt and Date
 - The details of books that will be purchased
 - Also, you can remove individual items from the Invoice as well as all invoice item

The interface features three tabs: "Books", "BookOrder", and "Order Details". Below the tabs is a search bar labeled "Enter Book Name". A "BookList" section contains a table with columns "BookTitle", "BookCost", and "QOH". The table has a header row and a data row with a "*" in the first column. Below the table is a button labeled "UpDate Stock" and a text input field. At the bottom, there are two buttons: "Clear All Rows" and "Remove Current Item", followed by another table with columns "BookTitle" and "BookCost". This table also has a header row and a data row with a "*" in the first column. To the right of this table is a button labeled "Record Sale". At the bottom center, there is a label "Total Amount" followed by a text input field.

	BookTitle	BookCost	QOH
*			

UpDate Stock

Clear All Rows

Remove Current Item

	BookTitle	BookCost
*		

Record Sale

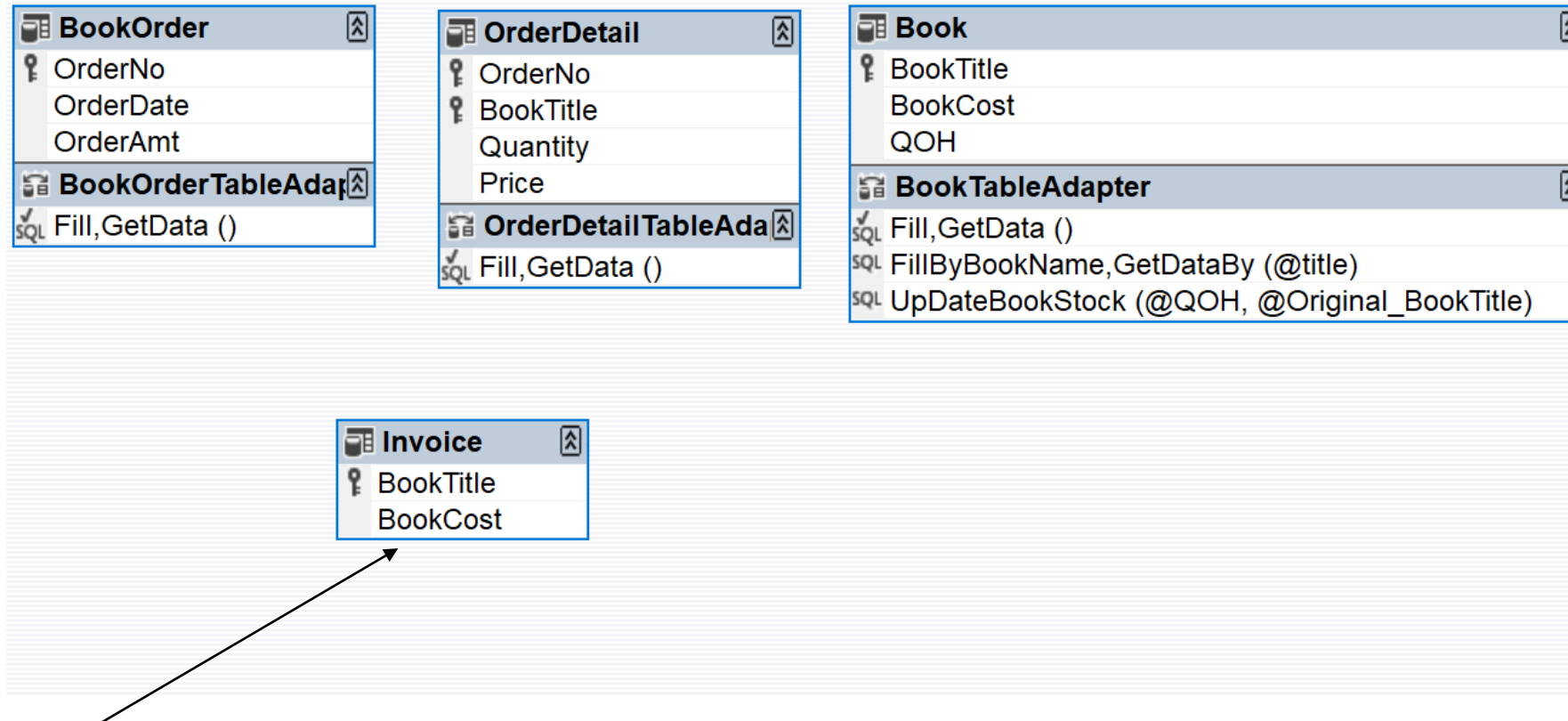
Total Amount

The Search and Update

- Update can be done in many ways
 - Via the Database
 - Via programming code
- We will write methods on the table adapter to:
 - Search for a Book
 - *SELECT BookTitle, BookCost, QOH*
 - *FROM Book*
 - *WHERE (BookTitle LIKE + '%' + @title + '%')*
 - Update the Stock of the book
 - *UPDATE Book*
 - *SET QOH = QOH + @QOH*
 - *WHERE (BookTitle = @Original_BookTitle);*

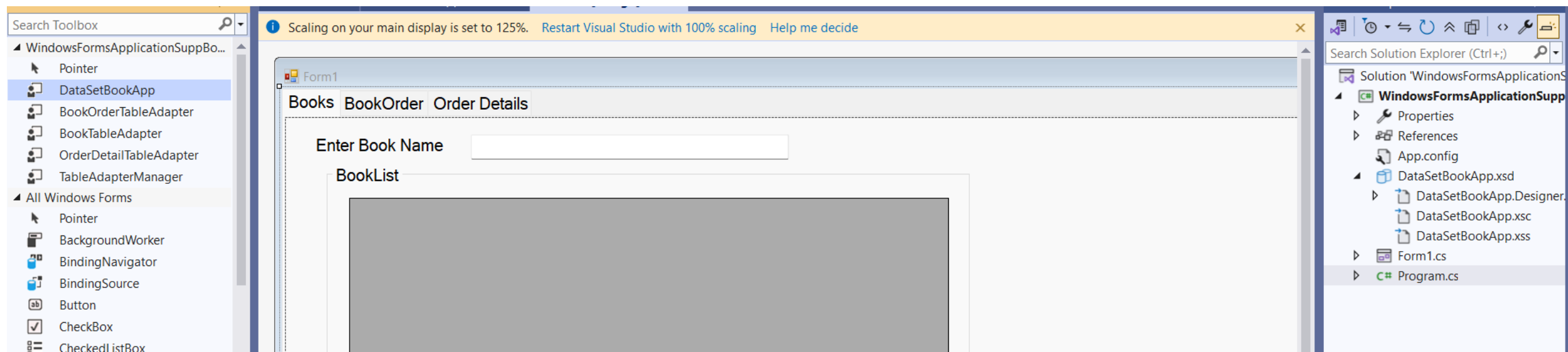
Book	
🔑	BookTitle
	BookCost
	QOH
BookTableAdapter	
✓ SQL	Fill, GetData ()
SQL	FillByBookName, GetDataBy (@title)
SQL	UpdateBookStock (@QOH, @Original_BookTitle)

An Invoice DataTable



- Add a DataTable to the DataSet and name it Invoice – add on 2 columns (string and decimal) - The BookTitle and BookCost data columns are added on – you need to ensure that the data types match those shown in the Book Table

Rebuild the Application to get the Data Adapters



- Use the adapters to generate adapter object that can be linked to the GridView

The Book Data and the Book Search

- The code required here is to fill the Gridview with Book Data and to provide a dynamic search option. The code is quite trivial because the SQL functionality has been written on the DataTable inside the DataSet.

```
private void Form1_Load(object sender, EventArgs e)
{
    TBook1.Fill(dsBookApp1.Book);
}

private void txtBookName_TextChanged(object sender, EventArgs e)
{
    TBook1.FillByBookName(dsBookApp1.Book, txtBookName.Text);
}
```


The Display and Search

Books BookOrder Order Details

Enter Book Name

BookList

	BookTitle	BookCost	QOH
▶	ASP3	975,00	43
	C# is Cool	605,00	10
	C# Journey	490,00	25
	DB in C#	675,00	36
	EF is Cool	675,00	14
	Java2	575,00	27
	Java3	795,00	15
*			

UpDate Stock

Clear All Rows

Remove Current Item

Total Amount

Record Sale

Books BookOrder Order Details

Enter Book Name

BookList

	BookTitle	BookCost	QOH
▶	C# Journey	490,00	25
	Java2	575,00	27
	Java3	795,00	15
*			

UpDate Stock

Let us also do the Update of Stock

- The functionality has already been created on the DataTable (See slide 4) – we simply have to invoke the method and the update will take place
- Also, once the update occurs, you have to refresh the gridview (by filling in all the rows again to see the updated effect)

```
private void btnStock_Click(object sender, EventArgs e)
{
    taBook1.UpdateBookStock(Convert.ToInt32(txtStockAmt.Text), dgvBook.CurrentRow.Cells[0].Value.ToString());
    taBook1.FillByBookName(dsBookApp1.Book, txtBookName.Text);
}
```

- The update logic is that for the current row, the QOH will be increased/decreased by the amount entered (-ve amount means decrease) – so if you enter 5 when the cursor is placed in the row for the book named “C# Journey”. then the QOH will change to 30 and the Gridview updates and shows the new value

Enter Book Name

BookList

	BookTitle	BookCost	QOH
	ASP3	975,00	43
	C# is Cool	605,00	10
▶	C# Journey	490,00	25
	DB in C#	675,00	36
	EF is Cool	675,00	14
	Java2	575,00	27
	Java3	795,00	15
*			

UpDate Stock

Enter Book Name

BookList

	BookTitle	BookCost	QOH
▶	ASP3	975,00	43
	C# is Cool	605,00	10
	C# Journey	490,00	30
	DB in C#	675,00	36
	EF is Cool	675,00	14
	Java2	575,00	27
	Java3	795,00	15
*			

UpDate Stock

Update the Stock of the book
UPDATE Book
SET QOH = QOH + @QOH
WHERE (BookTitle = @Original_BookTitle);

Adding Books to an Invoice

- Remember that an Invoice is simply a temporary table – once a transaction is recorded the Invoice Data Table must be cleared up completely
- The idea is that the user selects a book (by double clicking the cell content where the book is named – you could also use the row double click as well) and the book is added to the Invoice Table (for this app we will assume that the user can only select a book once only – i.e. the user can select many books but only 1 of each – code is shown below)
- Also whenever a book is selected the amount owing will be updated

```
private void dgvBook_CellContentDoubleClick(object sender, DataGridViewCellEventArgs e)
{
    DataRow dr;
    dr = dsBook1.BkInvoice.NewRow();
    for (int i = 0; i < dr.ItemArray.Length; i++)
    {
        dr[i] = dgvBook.CurrentRow.Cells[i].Value;
    }
    dsBook1.BkInvoice.Rows.Add(dr);
    ////txtAmtOwing.Text = getTotal().ToString("C2");
    //txtAmtOwing.Text = getTotal().ToString();
}
```

Enter Book Name

BookList

	BookTitle	BookCost	QOH
	ASP3	975,00	43
	C# is Cool	605,00	10
	C# Journey	490,00	30
▶	DB in C#	675,00	36
	EF is Cool	675,00	14
	Java2	575,00	27
	Java3	795,00	15
*			

UpDate Stock

Clear All Rows

Remove Current Item

	BookTitle	BookCost
▶	C# .Journey	490,00
	DB in C#	675,00
*		

Total Amount

A Better Version of adding rows to the dynamic Invoice Gridview

- The RowHeaderMouseDoubleClick is a more responsive event handler
- Instead of using a loop to iterate through the columns of the current row, you can use the actual names of the columns to fill up the “new row” with records from the datagridview
- The code below demonstrates this!

```
private void dgvBook_RowHeaderMouseDoubleClick(object sender,
DataGridViewCellMouseEventArgs e)
{
    DataRow dr;
    dr = dsBook1.BkInvoice.NewRow();
    dr["BookTitle"] = dgvBook.CurrentRow.Cells[0].Value;
    dr["BookCost"] = dgvBook.CurrentRow.Cells[1].Value;
    dsBook1.BkInvoice.Rows.Add(dr);
    txtAmtOwing.Text=getTotal().ToString("C2");
}
```

Calculate the Total for the Invoice

- This code entails iteration through the rows of the Invoice gridview and adding up the sub-totals

```
private double getTotal()
{
    double sum = 0;
    for (int i = 0; i < dsBookApp1.Invoice.Rows.Count; i++)
    {
        sum += Convert.ToDouble(dsBookApp1.Invoice.Rows[i][1]);
    }
    return sum;
}
```

Clearing Rows from the Invoice

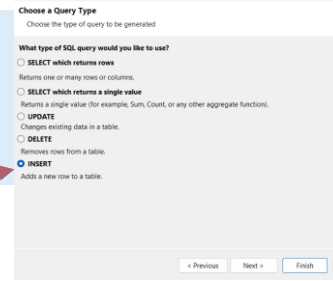
- To clear the current Invoice Row....

```
private void btnClearCurrentRow_Click(object sender, EventArgs e)
{
    dsBookApp1.Invoice.Rows[dgvBookInvoice.CurrentRow.Index].Delete();
    txtAmtOwing.Text = getTotal().ToString("C2");
}
```

- To clear all the Invoice Rows....

```
private void btnClearRow_Click(object sender, EventArgs e)
{
    dsBookApp1.Invoice.Rows.Clear();
    txtAmtOwing.Text = "";
}
```

Recording a Sale Transaction



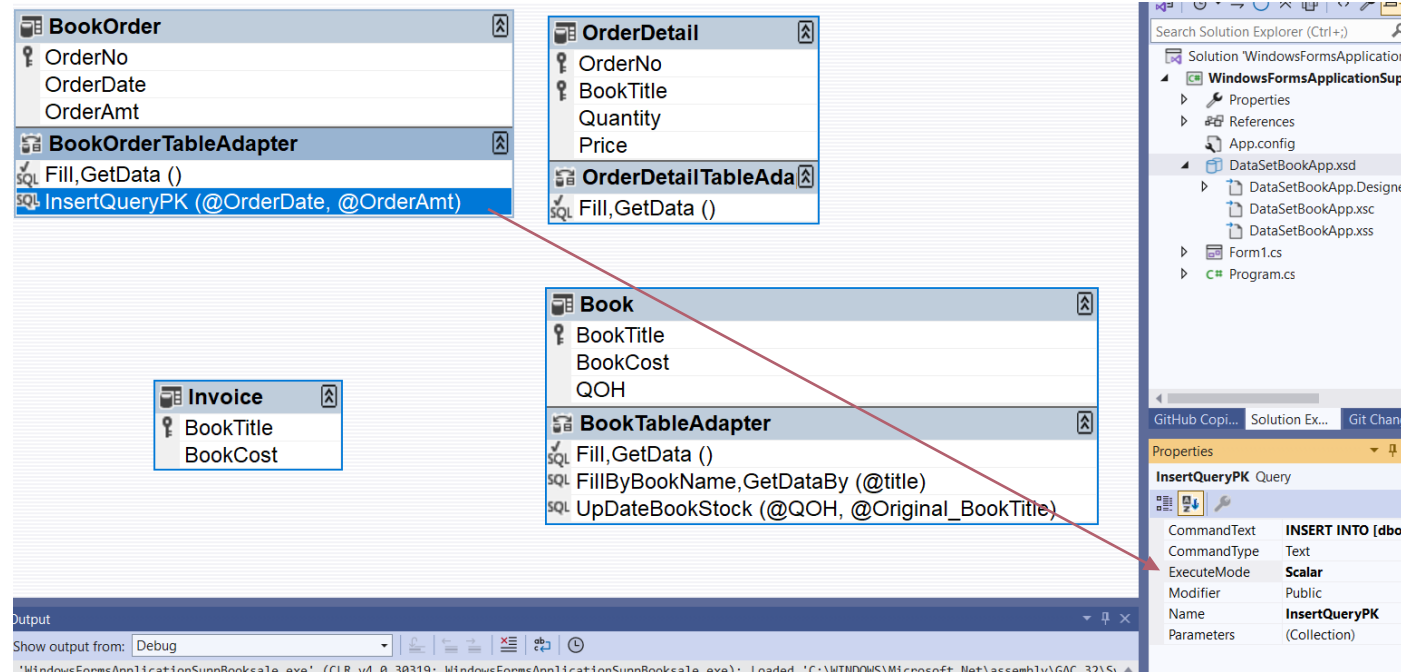
- There are 2 parts
- Part One is easy – this simply entail recording a Sale into the Order table with the overall Amount, Date and the Order ID that will be system generated
 - Once this is done the newly inserted OrderID must be returned so that you can record the Order details using this ID
 - If you right click look at the Order Table adapter, you can add a Sql Insert query; it has an Insert Sql statement that looks like

```
INSERT INTO [dbo].[BookOrder] ([OrderDate], [OrderAmt]) VALUES (@OrderDate, @OrderAmt);  
SELECT OrderNo, OrderDate, OrderAmt FROM BookOrder WHERE (OrderNo = SCOPE_IDENTITY())
```

- This statement does one of 2 things – it inserts a new Order into the BookOrder table; OR it inserts a new Order into the BookOrder table and returns the PK (OrderNo) of the newly inserted record. This is done via the Scope_Identity() function that you see in the Sql statement
- We will use the 2nd option where the Insertion takes place and the PK of the newly inserted record is returned

The Configuration

- On the InsertQuery of the Order Table Adapter you have to make a few changes:
 - Change the Execute mode to Scalar



- This will ensure that the `Scope_Identity ()` function kicks in and returns the PK of the newly inserted record; in the code, you will need to cast (convert) this value to an integer

The Code to Record the Main Transaction AND the Inner Details - Explanation

- The logic goes like this:
 - The Sale Order consists of many books
 - First Record the Overall Order
 - Return the PK of the newly Recorded/inserted order from the BookOrder table
 - Use this PK to record the details of all the books that formed part of the BookOrder; these details are recorded in the BookOrderDetails table; the PK for each row is a composite PK consisting of a combination of the PK of the overall Order and BookNo of each book that is part of the Order
 - Since there is usually more than one book in the Order, the recording of each book is done via a looping structure (such as a for loop)

The Code to Record a Sale and return the PK of the New Sale/Order record ...and a for loop to record the Inner Detail of the Order

```
private void btnRecordSale_Click(object sender, EventArgs e)
{
    int pk;
    DialogResult result = MessageBox.Show("Do you want to Confirm?", "Confirmation", MessageBoxButtons.YesNoCancel);
    if (result == DialogResult.Yes)
    {
        pk = (int)taBookOrder1.InsertQueryPK(DateTime.Now, Decimal.Parse(txtAmtOwing.Text, NumberStyles.Currency));
        MessageBox.Show("The Order for a Total Amount of " + txtAmtOwing.Text + "has been Confirmed and the PK is " + pk);
        taBookOrder1.Fill(dsBookApp1.BookOrder); //to fill the gridview with the newly recorded order so that it can be viewed
        for (int i = 0; i < dgvBookInvoice.Rows.Count - 1; i++) //record the inner detail of the Order
        {
            string bkID = dgvBookInvoice.Rows[i].Cells[0].Value.ToString();
            decimal bkPrice = (decimal)dgvBookInvoice.Rows[i].Cells[1].Value;
            taBookOrderDetail1.Insert(pk, bkID, 1, bkPrice); //the default quantity is 1
        }
    }
}
```

Things to Take Note of

- When a Sale is recorded – the PK is returned...An InsertQuery is executed where the Insert Parameters are the date and the Amount owing – note that `Decimal.Parse(txtAmtOwing.text, NumberStyle.Currency)` statement is used to extract/remove the currency symbol before the insertion into the database is made – that is because the AmountOwing field is a Decimal – not a String
- `pk = (int)taBookOrder1.InsertQueryPK(DateTime.Now, Decimal.Parse(txtAmtOwing.Text, NumberStyles.Currency));`
- This PK is used to record the inner detail of the Order – i.e.the details of each book is recorded together with the PK of the Order...
- Since there is one or more items to record as part of the inner details, a for loop is required...
- `for (int i = 0; i < dgvBookInvoice.Rows.Count - 1; i++)`
- The for loop iterates through all the rows of the Invoice Grdiview/DataTable and inserts each row into the Database...
- `taBookOrderDetail1.Insert(pk, bkID, 1, bkPrice);` //the assumption is made that the quantity for the purchase is 1 – we can always change this at a later stage...

The Visual Sequence

- Double click book rows for selection → Fill up the Invoice → Get the Confirmation → A message to show Sale details appear → The BookOrder shows details of the New Order → Click on the Row of the Order → and the INNER DETAILS ARE SHOWN IN THE GRIDVIEW BELOW!!

Enter Book Name

BookList

	BookTitle	BookCost	QOH
	ASP3	975,00	43
	C# is Cool	605,00	10
	C# Journey	490,00	30
▶	DB in C#	675,00	36
	EF is Cool	675,00	14
	Java2	575,00	27
	Java3	795,00	15
*			

UpDate Stock

Clear All Rows

Remove Current Item

	BookTitle	BookCost
▶	C# is Cool	605,00
	DB in C#	675,00
*		

Total Amount

R1 280,00

Confirmation

Do you want to Confirm?

Yes

No

Cancel

The Order for a Total Amount of R1 280,00has been PK is 34

	OrderNo	OrderDate	OrderAmt
	31	2024/05/31 06:16	R1 165,00
	32	2025/04/24 15:28	R1 285,00
	33	2025/05/15 17:40	R1 165,00
▶	34	2025/05/16 23:45	R1 280,00
*			

	OrderNo	OrderDate	OrderAmt
	31	2024/05/31 06:16	R1 165,00
	32	2025/04/24 15:28	R1 285,00
	33	2025/05/15 17:40	R1 165,00
▶	34	2025/05/16 23:45	R1 280,00
*			

	OrderNo	BookTitle	Quantity	Price
▶	34	C# is Cool	1	R605,00
	34	DB in C#	1	R675,00
*				

Showing the Inner Details of the Order

- This is simply a matter of configuration
- The 1st Gridview is linked to the Order Table and filled when the form loads..... `taBookOrder1.Fill(dsBookApp1.BookOrder);`
- The 2nd Gridview is linked to the OrderDetail Table and only gets filled when the user clicks on a row in the 1st Gridview

	OrderNo	OrderDate	OrderAmt
*			

	OrderNo	BookTitle	Quantity	Price
*				

```
private void dgvOrder_CellClick(object sender, DataGridViewCellEventArgs e)
{
    taBookOrderDetail1.FillByOrderNo(dsBookApp1.OrderDetail, (int)dgvOrder.CurrentRow.Cells[0].Value);
}
```

dsBookApp1 taBook1 bookBindingSource invoiceBindingSource taBookOrder1
orderDetailBindingSource

Record of Inner Details is Complete

- What if you wanted to enable the user to purchase a quantity greater than one
- You can change the Invoice table by adding in a Quantity column
- The old Invoice table ...and the new one

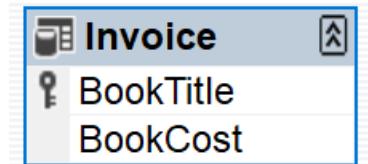


Diagram of the old Invoice table structure. It shows a table with two columns: BookTitle and BookCost. A key icon is next to BookTitle, indicating it is the primary key.

Invoice
BookTitle
BookCost

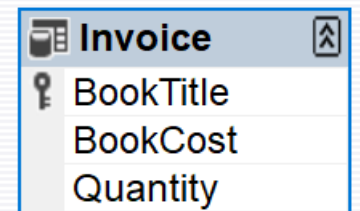
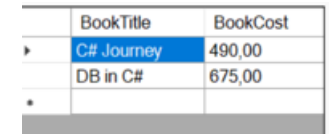


Diagram of the new Invoice table structure. It shows a table with three columns: BookTitle, BookCost, and Quantity. A key icon is next to BookTitle, indicating it is the primary key.

Invoice
BookTitle
BookCost
Quantity

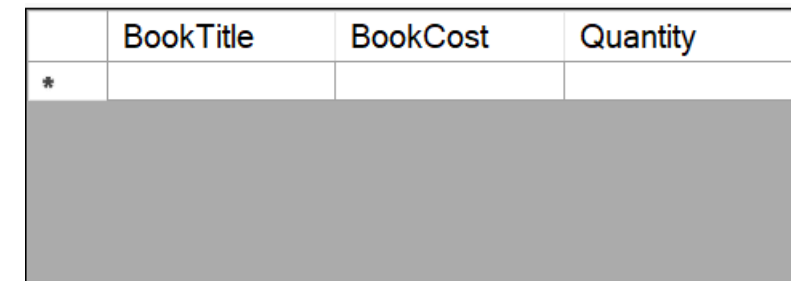
- Rebuild the application...Reset the data source for the Invoice Gridview and the Invoice Gridview changes from



Screenshot of the old data source. It shows a table with two columns: BookTitle and BookCost. The data rows are C# Journey (490,00) and DB in C# (675,00).

BookTitle	BookCost
C# Journey	490,00
DB in C#	675,00

- To a Gridview with a Quantity column



Screenshot of the new data source. It shows a table with four columns: BookTitle, BookCost, and Quantity. The data rows are C# Journey (490,00) and DB in C# (675,00). The Quantity column is currently empty.

	BookTitle	BookCost	Quantity
*			

Adding rows to Invoice Gridview with **Quantity**

- When a Book is chosen, the Book Row is added to the Invoice and the Quantity is set to 1
- This value can be edited and changed to any quantity value that is preferred...also you need to update the Total calculation by including the Quantity field
- Adjustments are required for the code record the sale as well because the quantity value has to be included

```
private double getTotal()
{
    double sum = 0;
    for (int i = 0; i < dsBookApp1.Invoice.Rows.Count; i++)
    {
        sum += Convert.ToDouble(dsBookApp1.Invoice.Rows[i][1])* Convert.ToInt16(dsBookApp1.Invoice.Rows[i][2]);
    }
    return sum;
}
```



The code to record the Order with Quantity Included

```
private void btnRecordSale_Click(object sender, EventArgs e)
{
    int pk;
    DialogResult result = MessageBox.Show("Do you want to Confirm?", "Confirmation",
    MessageBoxButtons.YesNoCancel);
    if (result == DialogResult.Yes)
    {
        pk = (int)taBookOrder1.InsertQueryPK(DateTime.Now, Decimal.Parse(txtAmtOwing.Text, NumberStyles.Currency));
        MessageBox.Show("The Order for a Total Amount of " + txtAmtOwing.Text + "has been Confirmed and the PK is " + pk);
        taBookOrder1.Fill(dsBookApp1.BookOrder); //to fill the gridview with the newly recorded order so that it can be viewed
        for (int i = 0; i < dgvBookInvoice.Rows.Count - 1; i++)//record the inner detail of the Order
        {
            string bkID = dgvBookInvoice.Rows[i].Cells[0].Value.ToString();
            decimal bkPrice = (decimal)(dgvBookInvoice.Rows[i].Cells[1].Value);
            int bkQty = Convert.ToInt32(dgvBookInvoice.Rows[i].Cells[2].Value);
            taBookOrderDetail1.Insert(pk, bkID, bkQty, bkPrice);//the assumption is made that the quantity for the purchase is 1 - we can always
            change this at a later stage...
        }
    }
}
```